

Video Processing Platform - System Architecture

Overview

This is an annotation platform designed to analyze videos containing structured content. The system automatically transcribes video audio, extracts discrete episodes based on delimiter words, and generates visual captions using advanced Vision Language Models. The architecture is modular to scale components independently.

Architectural Design Principles

Layered Architecture Pattern

The system follows a clean layered architecture with clear separation of concerns:

- **Presentation Layer:** React-based UI with server-side rendering
- **Gateway Layer:** Reverse proxy handling routing and SSL
- **Application Layer:** Business logic and API orchestration
- **AI Processing Layer:** External AI service integrations (OpenAI, Google Gemini)
- **Data Layer:** Persistent storage and analytics database

Key Design Patterns

Service-Oriented Architecture (SOA)

- Microservice-like separation of video processing, episode management, and analytics
- Each service handles specific domain responsibilities
- Loose coupling between components

Event-Driven Processing

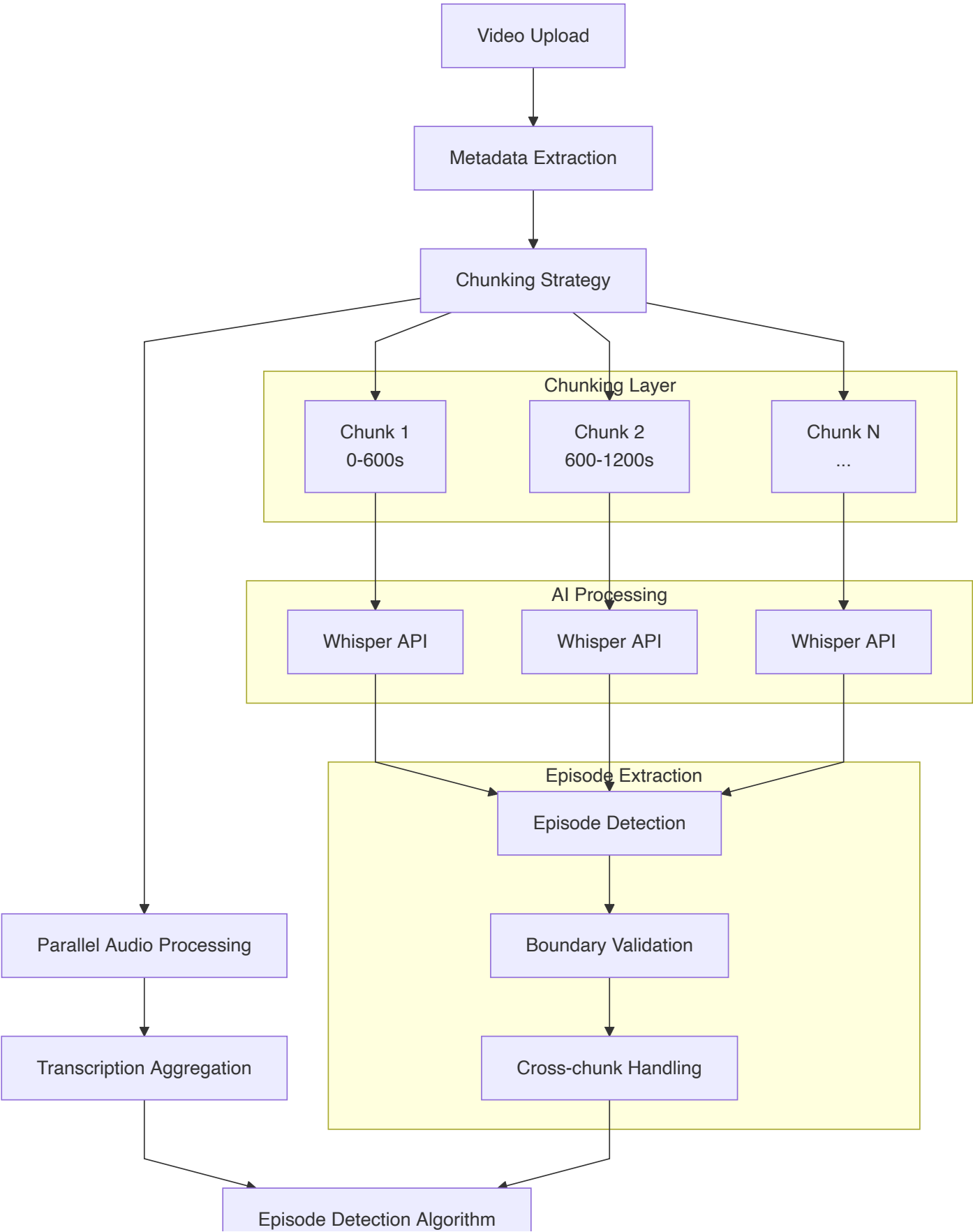
- Asynchronous video processing workflows
- Background task execution for resource-intensive operations
- Real-time status updates and progress tracking

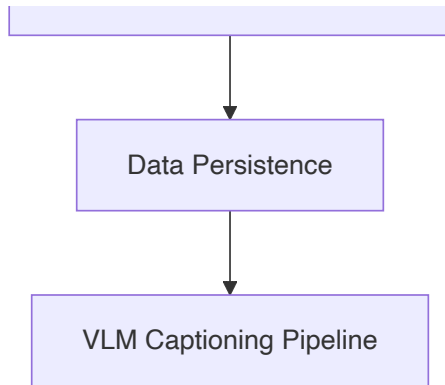
Caching Strategy

- Intelligent caching of transcription results to avoid reprocessing
- Hash-based cache keys for video segments

Video Processing Architecture

Core Processing Pipeline



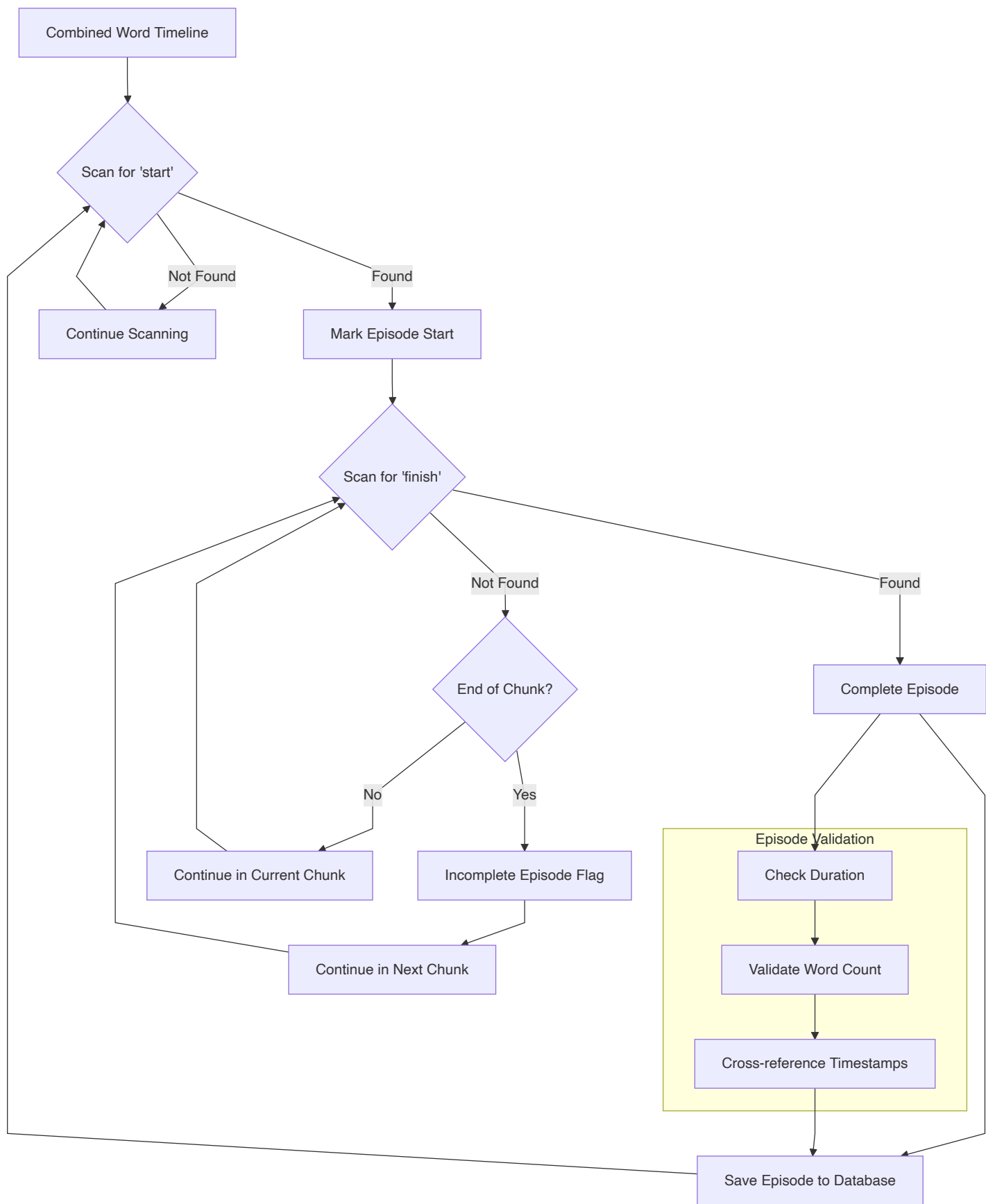


Video Chunking Strategy

The system employs an intelligent chunking strategy to handle videos of arbitrary length while maintaining processing efficiency and memory constraints. Chunking video will help in scaling the service to handle multiple episodes in parallel.

Episode Extraction Algorithm

The core innovation lies in the intelligent episode detection algorithm that can handle delimiter words spanning across multiple chunks.



Data Architecture

Database Design (ClickHouse)

The system uses ClickHouse as its primary database due to its exceptional performance with time-series data, analytics queries, and JSON metadata support.

VIDEOS		
String	id	PK
String	filename	
String	file_path	
UInt64	file_size	
Float64	duration_seconds	
Enum	processing_status	
String	metadata	
DateTime	created_at	

contains



processed_as



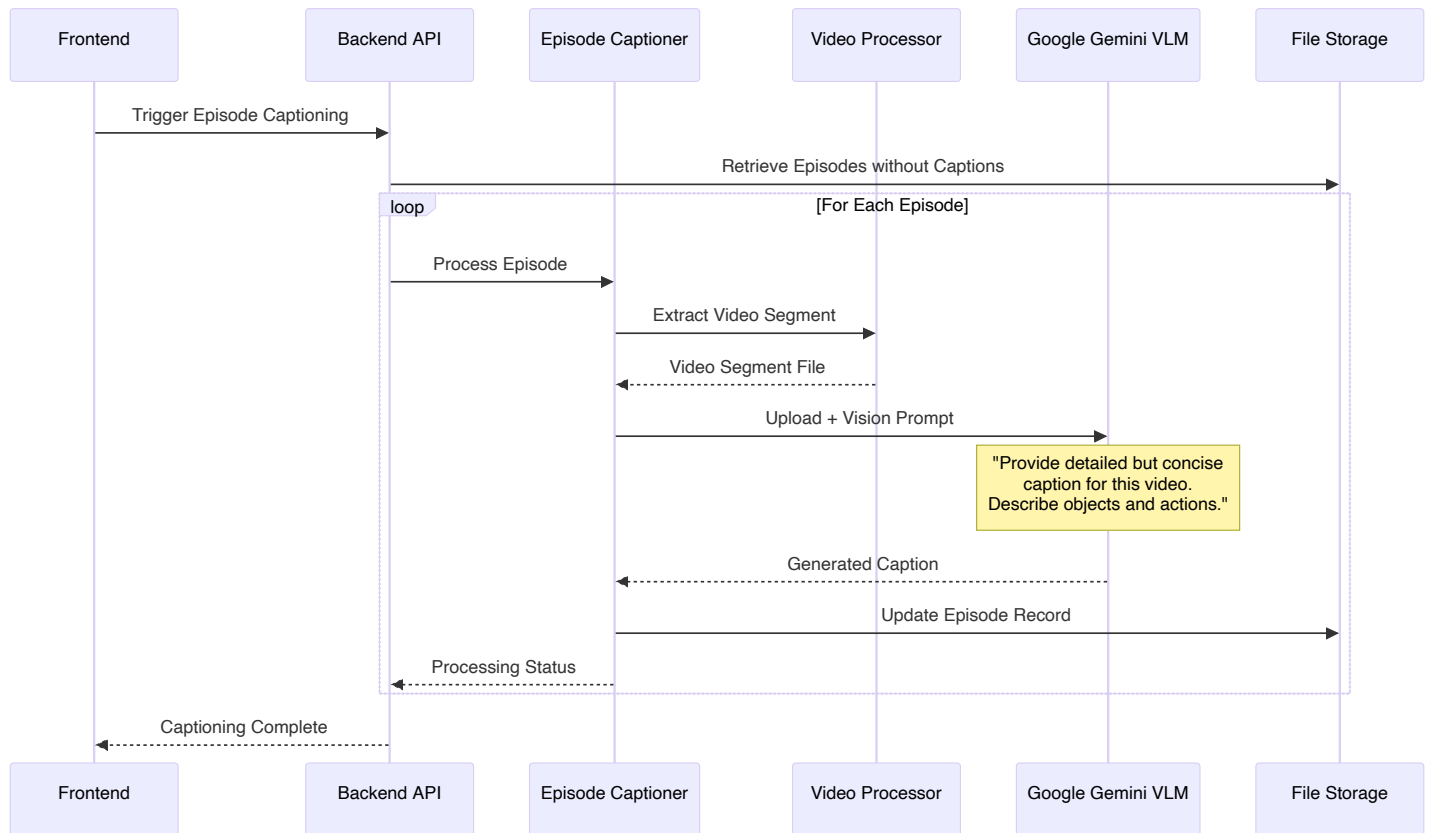
EPISODES		
String	id	PK
String	video_id	FK
UInt32	episode_number	
Float64	start_time	
Float64	end_time	
String	transcript	
String	captions	
String	metadata	

PROCESSING_CHUNKS		
String	id	PK
String	video_id	FK
UInt32	chunk_number	
Float64	start_time	
Float64	end_time	
String	cache_key	
UInt8	cache_hit	

Vision Language Model Integration

VLM Architecture Pattern

The system integrates Google Gemini VLM to generate visual captions for extracted video episodes, providing a complete multimodal understanding of the content.



VLM Processing Strategy

Smart Episode Selection:

- Skip episodes with existing captions
- Prioritize recent or high-importance episodes
- Batch processing with rate limiting

Error Resilience:

- Automatic retry mechanisms
- Fallback handling for API failures

Resource Management:

- Temporary file cleanup
- Memory-efficient video segment processing

- API quota management

API Design

RESTful Architecture

The system follows RESTful principles with clear resource-oriented endpoints:

Resource	Operations	Purpose
Videos	GET, POST	Video management and processing
Episodes	GET, PUT	Episode retrieval and annotation
Analytics	GET	System metrics and insights
Captioning	POST	VLM visual caption generation

API Characteristics

- **Async Processing:** Long-running operations handled via background tasks
- **Pagination Support:** Efficient data retrieval for large datasets
- **Status Tracking:** Real-time processing status updates
- **Error Handling:** Comprehensive error responses with actionable messages

Performance & Scalability

Architectural Optimizations

Connection Pool Management

- Thread-safe database connection pooling
- Configurable pool sizes and connection lifecycles
- Automatic connection cleanup and recovery

Intelligent Caching Strategy

- Hash-based cache keys for transcription segments
- Avoid reprocessing identical video chunks
- Memory-efficient cache management

Asynchronous Processing

- Background task execution for CPU-intensive operations
- Non-blocking API responses

- Real-time status updates through database polling

Scalability Design

Horizontal Scaling Capabilities

- Stateless API design enables load balancing
- ClickHouse supports distributed clustering
- Configurable storage backends (local, cloud)

Resource Management

- Chunked processing prevents memory overflow
- Configurable processing limits
- Smart episode selection for VLM processing

Technology Stack Overview

Layer	Technology	Key Benefits
Frontend	Next.js, TypeScript	Server-side rendering, type safety
Backend	FastAPI, Python	High performance async API
Database	ClickHouse	Analytics-optimized, time-series data
AI/ML	OpenAI Whisper, Google Gemini	Speech-to-text, vision understanding
Infrastructure	Nginx, PM2, Docker-ready	Production scalability

Architectural Design Choices

Intelligent Episode Detection

- **Cross-chunk Boundary Handling:** Episodes can span multiple processing chunks
- **Incomplete Episode Management:** Handles missing start/end delimiters gracefully
- **Timestamp Synchronization:** Maintains accurate timing across chunk boundaries

Multimodal AI Integration

- **Dual AI Pipeline:** Combines audio transcription with visual understanding
- **Smart Processing:** Skip logic prevents redundant VLM processing
- **Resource Optimization:** Efficient video segment extraction and cleanup

Scalable Data Architecture

- **Caching Strategy:** Hash-based transcription result caching
- **Analytics Optimization:** ClickHouse enables complex real-time queries

Video Streaming

- Video for each episode is streamed at low resolution for annotation
- Prefetching next episode

Annotations

- Each episode can be marked as a success or failure
- Caption/Description of a video can be updated

Future Architecture Considerations

Annotations

- Allow updating episode boundary for refining an episode
- Annotate incomplete or miss-detected episodes

Scalability Enhancements:

- Message queue integration (Redis/RabbitMQ) for new video or asynchronous episode processing

Dataset Management:

- Materialize episodes into individual video files
- Add motor sequences with each episode and create a multimodal dataset
- For each episode assign tags for filtering or training on specific events/episodes

Analytics/Enrichment:

- Assign episode quality (blurry video, incomplete video)

Operational Excellence:

- Comprehensive monitoring and alerting
- Performance metrics and optimization